

CS171 Project

Data Preprocessing

Author - Helena Thiessen

Project Objective

In a world that is seeing a rapid increase in waste production and feeling the effects of environmental damage from pollution, it is more important than ever to recycle and create a cleaner earth. Most drink containers are recyclable and yet only 43% of aluminum cans, 40% of glass bottles, and 20% of PET plastic bottles are recycled in the United States. This project aims to create a **Regional Convolutional Network** (RCNN) model that can **detect** recyclable drink containers within an image. In a real world application, this will allow the drink containers to be identified so they can be removed for recycling.

Dataset Overview

This project will be using the **Drinking Waste Classification** dataset sourced from [Kaggle](#). This dataset contains 4820 images of drink containers across 4 categories. The included classes are: Aluminum Cans, Glass bottles, PET (plastic) bottles and HDPE (plastic) Milk bottles. The dataset contains YOLO style labels for each image with classes being labelled from 0-3. The images and labels are all stored in one folder together with each image and its corresponding label having the same filename and appropriate extension. This dataset is ready for use and requires no additional preprocessing. The images are uncluttered and each contains only one drink container although there are variations in background, lighting, and positioning of containers. This raises the question: Can this dataset be used to create a model that can identify recyclable drinking containers in more diverse and real world scenarios. To test this, a validation set is manually created with images that contain multiple containers as well as other objects in scenes that resemble real life applications.

Data Prepration

In order to work with the FasterRCNN model, data must be transformed into a **custom Pytorch Dataset** object that can be used to create a dataloader. To do this I have created a **RCNN_Data Class** that inherits from Dataset and generates a dataset object when given a directory of the data location and a list of class names. This dataset expects the data to have the same storage structure present in the drinking waste classification dataset. The class then processes all of the images, properly defining the bounding boxes of each detection and

creating a target dictionary for each image. The resulting class is then ready to be imported and used in place of a **Pytorch Dataset** when creating the dataloaders that will be used for the RCNN model.

To create the validation set, I gathered up some aluminum cans, glass bottles, and PET bottles from my recycling and took images in different locations, with different backgrounds and arrangements of objects. The I found some images with milk bottles online to add to this. Next, I used a script to rename them all to **standardized naming** and **resize** them to be more aligned with the test and training data. I then used MakeSense to **draw bounding boxes** for each image so that I could later compare the actual and predicted boxes for the objects in the images. MakeSense generated files of the correct format that I was able to add to my image folder so my data was ready to use.

Helper tools then need to be created to display the data. To draw meaningful conclusions from the model results it is necessary to be able to display the bounding boxes on top of the image. To accomplish this, I created the function **draw_boxes** which takes a batch of images and targets and optionally predicted targets and displays the images with actual targets in green and predicted targets in red. The predicted targets can be shown with a **Non-Maximum Suppression** (nms) filter applied that filters out overlapping boxes based on a supplied threshold and a probability filter to only show boxes above a given probability.

1. Project Setup

- Import Necessary Libraries
- Set device for appropriate Pytorch use

```
In [3]: import torch
from torch.utils.data import Dataset
from torchvision.io import read_image
import numpy as np
import os
import glob
import cv2
from pathlib import Path
import matplotlib.pyplot as plt
from natsort import natsorted
from torchvision.ops import nms
from torchvision import transforms
from PIL import Image
import math
```

```
In [4]: #determine what device is in use
if torch.backends.mps.is_available():
    device = torch.device("mps")
elif torch.cuda.is_available():
    device = torch.device("cuda")
else:
```

```
device = torch.device("cpu")
print("Using device:", device)
```

Using device: cuda

2. Pre-Processing For Drinking Waste Classification Dataset

- Create a custom dataset class to represent drink container images and detection box labels

In [5]: *##A custom dataset for using drinking waste classification images in an RCNN*

```
class RCNN_Data(Dataset):
    def __init__(self, root_dir, transforms = None):
        """
        Creates an instance of RCNN_data
        Params:
        root_dir - the directory where the data is located
        transforms - the transforms being applied to the data
        """
        self.root_dir = str(root_dir)
        self.class_names = ["Aluminum", "Glass", "HDP", "PET"]
        self.transforms = transforms

        # pair images with annotations
        self.samples = []
        for fn in sorted(os.listdir(self.root_dir)):
            if fn.lower().endswith((".jpg", ".jpeg", ".png")):
                stem = os.path.splitext(fn)[0]
                txt = os.path.join(self.root_dir, stem + ".txt")
                self.samples.append((os.path.join(self.root_dir, fn), txt))

    def __len__(self):
        """
        Returns:
        The length of the dataset
        """
        return len(self.samples)

    def __getitem__(self, idx):
        """
        Params:
        idx - the index of the data item being retrieved

        Returns:
        An individual item and target
        """
        image_path, label_path = self.samples[idx]

        #Load image
        image = read_image(image_path).float() / 255.0
        _, H, W = image.shape
```

```

#read + process labels
boxes_t = torch.zeros((0,4), dtype=torch.float32)
labels_t = torch.zeros((0,), dtype=torch.int64)
if os.path.exists(label_path) and os.path.getsize(label_path) > 0:
    arr = np.loadtxt(label_path, ndmin=2, dtype=np.float32)
    if arr.ndim == 2 and arr.shape[1] >= 5:
        labels_np = arr[:, 0].astype(np.int64) + 1
        cxcywh_np = arr[:, 1:5].astype(np.float32)
        cx = cxcywh_np[:, 0] * W
        cy = cxcywh_np[:, 1] * H
        ww = cxcywh_np[:, 2] * W
        hh = cxcywh_np[:, 3] * H
        x1 = np.clip(cx - ww / 2, 0, W-1)
        y1 = np.clip(cy - hh / 2, 0, H-1)
        x2 = np.clip(cx + ww / 2, x1 + 1e-3, W-1)
        y2 = np.clip(cy + hh / 2, y1 + 1e-3, H-1)
        boxes_t = torch.from_numpy(np.stack([x1,y1,x2,y2],1)).float()
        labels_t = torch.from_numpy(labels_np).long()

#create target
target = {
    "boxes": boxes_t,
    "labels": labels_t,
    "image_id": torch.tensor([idx]),
    "area": (boxes_t[:,2]-boxes_t[:,0])*(boxes_t[:,3]-boxes_t[:,1]),
    "iscrowd": torch.zeros((boxes_t.shape[0],), dtype=torch.int64)
}

if self.transforms is not None:
    image, target = self.transforms(image, target)

return image, target

```

3. Tools for displaying Images and Targets

- draw_boxes outputs an image with the targets drawn on
- nms_filter filters out overlapping boxes

```

In [6]: def filter_one_pred(pred, iou_threshold=0.7, score_threshold=0.5):
        """
        Applies nms and probability filters to one prediction

        params:
        iou_threshold - the threshold value used for NMS filtering
        score_threshold - the threshold value used for probability filtering

        returns:
        filtered boxes, scores, and labels for a prediction
        """
        boxes = pred["boxes"]
        scores = pred["scores"]
        labels = pred["labels"]

```

```

# score / probability threshold
keep = scores >= score_threshold
boxes = boxes[keep]
scores = scores[keep]
labels = labels[keep]

if boxes.numel() == 0:
    return {"boxes": boxes, "scores": scores, "labels": labels}

# NMS
keep_idx = nms(boxes, scores, iou_threshold)

return {
    "boxes": boxes[keep_idx],
    "scores": scores[keep_idx],
    "labels": labels[keep_idx],
}

```

```

In [7]: def draw_boxes(images, targets, preds=None, iou_threshold=0.7, score_threshold=0.5)
        """
        Draw bounding boxes on an image. Can draw true only or true and predicted
        Predicted boxes can be filtered by IOU and probability

        Params:
        images - The images being displayed
        targets - A target dictionary with the actual labels
        preds - A target dictionary with the predicted labels
        iou_threshold - the threshold for NMS filtering
        score_threshold - the threshold for probability filtering
        """

        if preds is None:
            preds = [None] * len(images)

        n = min(len(images), len(targets), len(preds))

        cols = 2
        rows = math.ceil(n / cols)

        fig, axes = plt.subplots(rows, cols, figsize=(4 * cols, 4 * rows))
        axes = np.array(axes).reshape(-1)

        for i in range(n):
            ax = axes[i]

            img_t = images[i]
            tgt = targets[i]
            pred = preds[i]
            img = img_t.permute(1, 2, 0).numpy()
            img = (img * 255).astype(np.uint8)

            #draw true in green
            for box, lab in zip(tgt["boxes"], tgt["labels"]):
                class_id = lab.item()
                x1, y1, x2, y2 = map(int, box.tolist())
                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

```

```

cv2.putText(img,
            f"Class {class_id}",
            (x1, y1 - 5),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6, (0,255,0), 2)

# draw pred in red
if pred is not None:
    pred = filter_one_pred(pred, iou_threshold=iou_threshold, score_thresh=score_thresh)
    for box, lab in zip(pred["boxes"], pred["labels"]):
        class_id = lab.item()
        x1, y1, x2, y2 = map(int, box.tolist())
        cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)
        cv2.putText(img,
                    f"Class {class_id}",
                    (x1, y1 - 5),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.6, (255,0,0), 2)

ax.imshow(img)
ax.set_title(f"Image {i}")
ax.axis("off")

# hide any unused subplots
for j in range(n, len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()

```

4. Pre-processing for Validation Dataset

- Obtain images
- Format images correctly
- Create bounding boxes

Description of processing performed by hand

Obtaining Images

To create a validation dataset I needed images that contained plastic drink bottles, aluminum cans, glass bottles, and milk jugs. I wanted to use photos that were busier than the ones in the training and test sets and contained multiple detections in an image as well as overlapping boxes. For the plastic drink bottles, aluminum cans and glass bottles I was able to take photos of objects I had in my recycling which allowed me to get images with different mixes of these objects. For milk jugs I had to obtain my images from internet searches, making these images more limited in their variability.

Formatting Images

After running the code cells below, inspect images to ensure they are still usable and nothing unexpected happened while altering them.

Creating Bounding Boxes After downsizing and renaming images I used makesense.ai to draw YOLO format detection bounding boxes for each image so that the actual and predicted boxes could be compared when testing the model. This involved creating a bounding box for each instance of drink containers found in the validation images and selecting the appropriate label for each box. I then had to export these labels to the project folder and place them as my dataset class expects them. I now have a ready to use val set!

In [8]: *#rename images in val folder to have 'cleaner' consistent names
#this only needs to be run once for any new val images
#requires that val images have jpg extension*

```
def rename():
    """
    names files by increasing index
    replaces the existing file with the new file
    """
    root = Path.cwd() / "Images_of_Waste" / "val"

    files = os.path.join(root, f"*.{ 'jpg' }")
    files = glob.glob(files)
    files = natsorted(files)

    for i, path in enumerate(files):
        dir = os.path.dirname(path)
        new_name = f"val_{str(i + 1).zfill(2)}.jpg"
        new_path = os.path.join(dir, new_name)

        try:
            os.rename(path, new_path)

        except Exception as e:
            print(f"An error occurred: {e}")
```

In [9]: *#resize images to better align with test/train images
#set short side to 512, let it maintain AR
#this only needs to be run once for any new val images
#again set up to work specifically with jpg*

```
def resize():
    """
    Resizes images to 512 on the short side
    Replaces the existing file with the new file
    """
    root = Path.cwd() / "Images_of_Waste" / "val"
    resize = transforms.Resize(512)

    for item in os.listdir(root):
        path = os.path.join(root, item)
        if os.path.isfile(path):
            try:
                with Image.open(path) as img:
```

```
        resized = resize(img)
        resized.save(path, img.format)
        print(f"Resized and saved: {item}")
    except Exception as e:
        print(f"Could not process {item}: {e}")
```

```
In [10]: #rename()
        #resize()
```